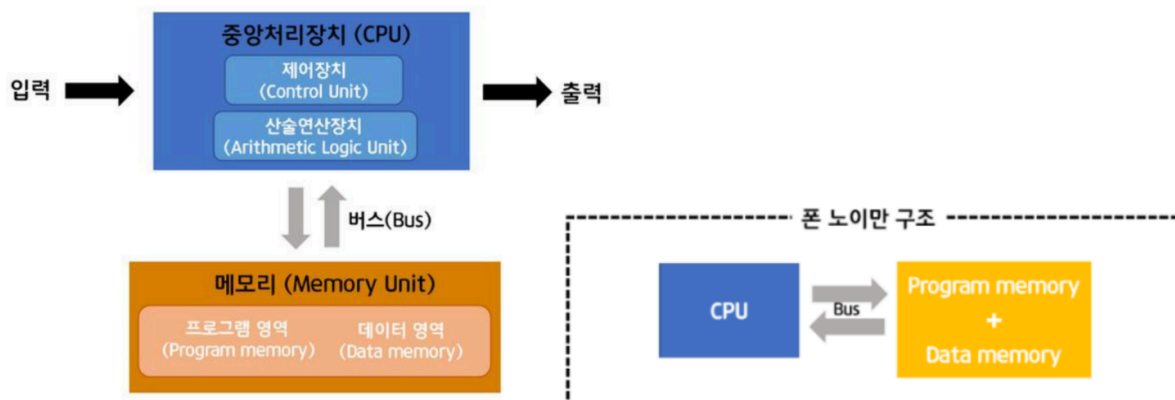


ROS2 기초 요약본

1. 컴퓨터 구조와 리눅스

1.1 폰 노이만 구조

- 중앙처리장치(CPU), 메모리, 프로그램 세 가지 요소로 구성



1.2 컴퓨터 부팅 과정



1.3 컴퓨터 구조

- **CPU란 무엇인가**
 - 컴퓨터의 뇌 ⇒ 중앙처리장치
 - 주 기억장치인 메모리에서 명령어를 읽어 들이고 이를 해석하여 수행하는 작업을 맡음
- **프로그램이란 무엇인가**
 - 명령어들의 집합
 - **순서대로 실행될 수 있도록 작성된 정적인 파일**

- **프로그래밍 언어**로 작성되며, **사용자 목적을 처리**할 수 있도록 구성됨
- 보조 기억장치(예: 하드디스크)에 저장되어 있음
- **프로세스(Process)란?**
 - 실행 중인 프로그램, 즉 **동적인 상태의 프로그램**
 - 각 프로세스는 **독립적인 메모리 공간**(Code, Data, Stack, Heap)을 가짐
 - **다른 프로세스와 메모리 공유 불가** → 접근하려면 **IPC(프로세스 간 통신)** 필요
 - **하나 이상의 스레드**를 포함함
- **스레드(Thread)란?**
 - **프로세스 내에서 실행되는 작업 단위**, 실행 흐름
 - **프로세스 자원을 공유하며 작업을 수행하는 실행 주체(일꾼)**
 - 프로세스는 최소 하나의 스레드(메인 스레드)를 가짐
- **멀티스레딩(Multi-threading)이란?**
 - 하나의 프로그램(프로세스) 안에서 **여러 스레드가 동시에 작업**을 수행하는 방식
 - **웹 서버** 같은 고성능 응용프로그램에 자주 사용
 - 많은 운영체제(윈도우, 리눅스 등)에서 기본적으로 지원

1.4 Application 작동원리

- **마이크로 프로세서**
 - 산술 논리 장치, 제어 장치, 레지스터를 하나의 단일체 집적회로로 구성한 것을 의미
- **메모리**
 - 코드(code) 영역
 - 실행할 프로그램의 코드가 저장되는 영역으로 텍스트(code) 영역이라고도 부름
 - 데이터(data) 영역
 - 프로그램의 전역 변수와 정적(static) 변수가 저장되는 영역이며, 프로그램의 시작과 함께 할당되며, 프로그램이 종료되면 소멸
 - 스택(stack) 영역
 - 힙(heap) 영역

- 사용자가 직접 관리해야 하는 메모리 영역
- 메모리 공간이 **동적으로** 할당 및 해제됨
- 메모리의 **낮은 주소 → 높은 주소** 방향으로 할당됨

1.5 리눅스와 운영체제

• 운영체제란?

- 사용자가 컴퓨터를 사용하기 위해 필요한 소프트웨어
- 컴퓨터의 하드웨어 관리 + 사용자 편의 제공

• 특징

- 오픈 소스 운영체제
 - 누구나 자유롭게 사용, 수정, 배포 가능
- 강력한 네트워크 지원
 - 서버 운영에 최적화되어 있어 웹서버, DB서버 등에 널리 사용
- 다중 사용자 및 다중 작업 지원
 - 여러 사용자가 동시에 시스템 사용 가능, 여러 작업 동시 처리
- 이식성과 확장성
 - 다양한 하드웨어와 플랫폼에서 동작 가능하며, 커스터마이징 용이

• 리눅스 기본 명령어

명령어	설명	비고 (영문 의미 등)
<code>pwd</code>	현재 디렉토리 경로 출력	print working directory (폴더 위치)
<code>ls</code>	현재 위치에서 파일/디렉토리 목록 확인	list
<code>ls -a</code>	숨김파일까지 확인	all
<code>ls -l</code>	상세 정보 표시 (권한, 용량 등)	<code>-al</code> 같이 조합 사용 가능
<code>cd [경로]</code>	디렉토리 이동	- <code>cd ..</code> : 상위 디렉토리 이동 - <code>cd .</code> : 현재 디렉토리 - <code>cd /</code> : 루트로 이동
<code>mkdir [이름]</code>	새 디렉토리 생성	make directory

<code>rmdir [이름]</code>	디렉토리 삭제 (비어있어야 함)	remove directory
<code>clear</code>	터미널 화면 내용 지우기	화면 초기화
<code>history</code>	사용한 명령어 기록 출력	숫자 입력 시 해당 명령 재실행 가능
<code>touch [이름]</code>	새 파일 생성	예: <code>touch index.html</code>
<code>cat [파일명]</code>	파일 내용 출력	concatenate (연결하여 출력)
<code>tail [파일명]</code>	파일 뒷부분의 몇 행을 출력	기본값은 10
<code>more [파일명]</code>	화면에 보이는 만큼만 출력	아래에 내용이 더 있으면 —more—(XX%)로 표시됨

• 디렉터리와 용도

디렉터리	용도
<code>/bin</code>	기본 실행 명령어가 위치
<code>/dev</code>	디바이스 파일이 위치
<code>/etc</code>	시스템 설정 파일이 위치
<code>/home</code>	사용자별 홈 디렉터리가 위치
<code>/lib</code>	시스템 라이브러리가 설치
<code>/mnt</code>	외부 디스크 등을 임시로 마운트할 때 사용
<code>/proc</code>	<code>procfs</code> 가 마운트됨 (가상 파일 시스템)
<code>/root</code>	<code>root</code> 사용자의 홈 디렉터리
<code>/sbin</code>	시스템 관리용 명령어가 위치
<code>/sys</code>	<code>sysfs</code> 가 마운트됨 (시스템 정보 제공)
<code>/tmp</code>	시스템 임시 파일이 위치
<code>/usr</code>	시스템 소프트웨어가 설치됨

• 리눅스 권한

- **root** : 시스템을 관리할 수 있는 최상위 관리자 계정
- **su (Switch User)** : 현재 계정을 로그아웃하지 않고 다른 계정으로 전환하는 명령어
- **sudo**
 - `superuser do` 의 약자
 - 일시적으로 관리자 권한을 부여하여 명령어를 실행할 수 있게 해줌

- 일반 사용자가 **시스템 파일 수정, 프로그램 설치** 등 관리자 작업을 할 수 있도록 허용
- 시스템 보호를 위해 **root 계정과 구분됨**

○ 권한 설정

- 리눅스에서는 누가 파일을 읽고, 쓰고, 실행할 수 있는지를 정함
- r(읽기), w(쓰기), x(실행) ex) `drwxr-xr-x`

위치	뜻	예시
1번째 글자	타입	<code>d</code> 면 디렉터리, <code>-</code> 면 일반 파일
2~4번째	소유자 권한	<code>rwX</code> → 읽기, 쓰기, 실행 다 가능
5~7번째	그룹 권한	<code>r-X</code> → 읽기, 실행 가능 (쓰기 ❌)
8~10번째	기타 사용자	<code>r-X</code> → 읽기, 실행 가능 (쓰기 ❌)

○ 리눅스 권한 수정 명령어

명령어	의미
<code>chmod u+w 파일명</code>	소유자에게 쓰기 권한 추가
<code>chmod g-w 파일명</code>	그룹에서 쓰기 권한 제거
<code>chmod o+x 파일명</code>	기타 사용자에게 실행 권한 추가
<code>chmod 755 파일명</code>	숫자로 권한 설정 (<code>rwxr-xr-x</code> 와 같음)

○ 리눅스 삭제 명령어 정리

설명	명령어
<code>test.txt</code> 파일 삭제	<code>rm test.txt</code>
<code>test.txt</code> 파일 강제 삭제 (문지 않고 즉시)	<code>rm -f test.txt</code>
확장자가 <code>.txt</code> 인 모든 파일 삭제	<code>rm *.txt</code>
현재 디렉터리 모든 파일 삭제 (디렉터리는 삭제되지 않음)	<code>rm *</code>
현재 디렉터리 모든 파일 강제 삭제 (문지 않고 즉시)	<code>rm -f *</code>
<code>test</code> 디렉터리 삭제	<code>rm -r test</code>
<code>test</code> 디렉터리 강제 삭제 (문지 않고 즉시)	<code>rm -rf test</code>
현재 디렉터리 모든 파일과 디렉터리 삭제	<code>rm -r *</code>
현재 디렉터리 모든 파일과 디렉터리 강제 삭제 (문지 않고 즉시)	<code>rm -rf *</code>

• 우분투 터미널

- 사용자가 컴퓨터에 명령어를 입력하여 시스템을 제어 관리할 수 있음
 - Ctrl + C: 현재 작업 중지
 - Ctrl + Z: 작업 일시 중지
 - Ctrl + D: 터미널 로그아웃 또는 종료
 - Tab: 자동 완성
 - Ctrl + R: 명령어 기록에서 검색

2. 네트워크와 통신

2.1 웹 통신 기초

- **API**
 - 응용 프로그램에서 다른 프로그램의 기능을 사용하도록 제공된 인터페이스
 - 클라이언트와 프로그램 사이를 연결하는 다리 역할
 - 정해진 방식으로 요청하면 필요한 기능이나 데이터를 제공
- **라이브러리(Library)**
 - 응용 프로그램 개발에 필요한 기능(함수)들을 모아놓은 소프트웨어
 - 주로 클래스, 함수, 미리 작성된 코드, 자료형, 도움말 등을 포함
 - **독립성**을 가짐 (다른 라이브러리에 의존하지 않음)
- **Framework**
 - 소프트웨어 개발을 쉽게 하기 위해 제공된 **개발 환경(뼈대, 기반 구조)**
 - 개발자는 프레임워크 위에서 **정해진 규칙에 따라** 코드를 작성하여 프로그램을 완성
 - 프레임워크가 응용 프로그램을 능동적으로 제어 (**개발자가 수동적으로 따름**)
- **HTTP**
 - 웹에서 클라이언트(브라우저)와 서버 간의 **통신 규칙(프로토콜)**
 - 데이터가 **일반 텍스트로 교환** (암호화되지 않음)
- **HTTPS**
 - HTTP의 **보안이 강화된 확장 버전**
 - 데이터가 **암호화된 상태로** 안전하게 교환

- 브라우저와 서버가 통신 전에 **보안 연결(SSL/TLS)** 설정
- **XML**
 - 데이터를 저장하고 공유하기 위한 마크업 언어
 - 웹사이트, 데이터베이스, 애플리케이션 간에 **구조화된 데이터**를 쉽게 교환할 수 있게 도와줌
 - ROS에서 XML의 역할 → ROS의 패키지 빌드 및 의존성 관리를 해줌

2.2 네트워크와 통신

- **인터넷 프로토콜(IP)**
 - 데이터 패킷이 네트워크를 통해 이동하고 올바른 대상에 도착할 수 있도록 데이터 패킷을 라우팅하고 주소를 지정하기 위한 프로토콜 또는 규칙의 집합
- **IPv4 vs IPv6**

항목	IPv4	IPv6
주소 길이	32비트	128비트
주소 표기	십진수 4개 (예: 192.168.0.1)	16진수 8개 (예: 2001:0db8:85a3::8a2e:0370:7334)
주소 개수	약 42억 개 (2^{32})	사실상 무한 (2^{128})
주소 유형	유니캐스트, 브로드캐스트, 멀티캐스트	유니캐스트, 멀티캐스트, 애니캐스트
브로드캐스트	지원함	지원하지 않음 (멀티캐스트 또는 애니캐스트 사용)

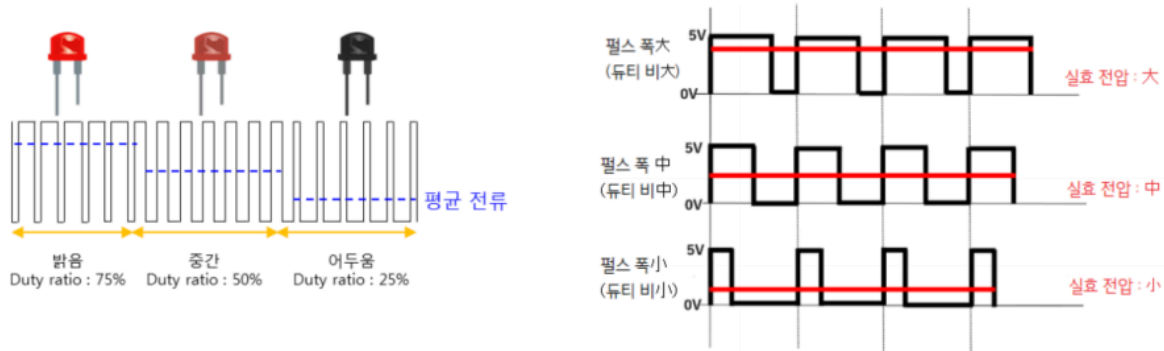
3. 센서와 로봇 기초

3.1 센서 기초

- 센서(Sensor)란?
 - 외부의 물리·화학적 정보를 감지하여 이를 전기 신호 등으로 변환해 전달하는 장치로, 인간의 감각과 유사한 역할을 하며 정보를 측정·검출하는 기능을 수행함.

- PWM 신호

- 특정 형태의 디지털 신호
- 아날로그 신호처럼 동작하는 효과를 디지털 방식으로 만들어냄



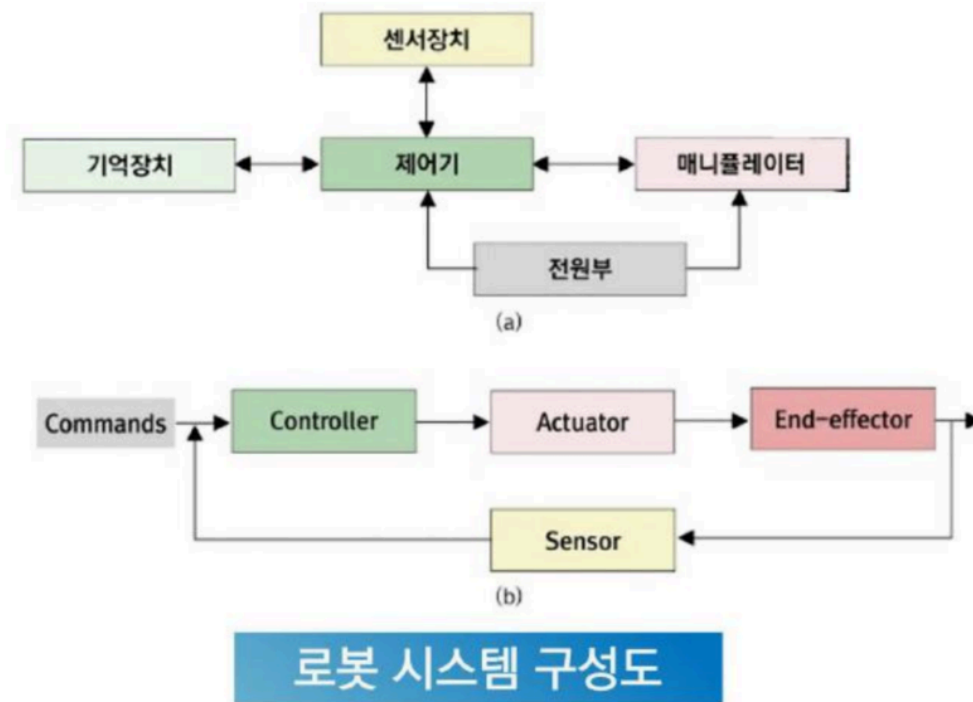
3.2 IoT(사물 인터넷)

- 연결된 디바이스의 공통 네트워크를 의미하며, 디바이스와 클라우드 및 디바이스 간 통신을 용이하게 하는 기술을 의미

3.3 임베디드(Embedded)

- 내장형 시스템은 특정 기능을 수행하도록 설계된, 디바이스 내부에 탑재된 컴퓨터 시스템
- 디바이스를 직접 제어하는 소프트웨어가 하드웨어에 내장되어 작동
- 메모리 조작이 핵심, C/C++과 같은 저수준 언어가 적합

3.4 로봇의 센서 활용 구성



3.5 로봇의 구성

- 기구부
 - 로봇 프레임, 하우징, 바퀴 및 바퀴 축, 베어링, 기어, 벨트, 말단효과장치 등
- 하드웨어부
 - 컴퓨터, 제어기, 전원관리 보드, 구동기(모터), 센서 등
- 소프트웨어부
 - 하드웨어 제어 프로그램(모터제어 프로그램, 센서 입출력 프로그램), 운영체제, 각종 응용 소프트웨어 등

3.6 로봇의 조인트

- 로봇을 움직이게 해주는 회전 또는 직선 운동을 담당하는 연결 부위
 - 직선 관절 - 한 방향으로 직선 이동만 가능
 - 회전 관절 - 한 축을 중심으로 회전만 가능
 - 평면 관절 - 평면상에서 2축 이동 + 1축 회전 가능
 - 구면 관절 - 3축 회전 가능

- 원통형 관절 - 직선 이동 + 회전
- 스크류 관절 - 나사처럼 회전 시 직선 이동이 동반됨

3.7 좌표계

- 직교 좌표, 데카르트 좌표계 (Cartesian Coordinate System)
 - 가장 기본적인 좌표계
 - 축: x, y, z
- 원통 좌표 좌표계 (r, θ, z), rho-theta
 - X, Y 축에서 원점과 해당 좌표까지의 거리(r)와 해당 점에서 원점으로 이은 선이 X 축과의 이루는 각(θ)으로 좌표를 표시한다. (z축은 동일)
- 구면 좌표계 (r, θ, ϕ)
 - 구 형태의 공간에서 점의 위치를 나타냄
 - 보통 회전 운동이나 구 형태의 경로 계산에 유리함

4. ROS2

4.1 ROS2란?

- 로봇을 위한 **오픈 소스 메타 운영 체제**
- **ROS의 주요 기능**
 - 로봇 **하드웨어 추상화**
 - **프로세스 간 통신** 지원 (노드 간 메시지 전달 등)
 - **패키지 관리 시스템**을 통한 코드 구성 및 공유
 - 다양한 오픈소스 모듈과의 연결을 통한 **확장성**
- **ROS의 구성요소**

구성요소	설명
마스터	- 노드와 노드 사이 연결 및 메시지 통신의 네임서버 역할 - roscore 를 실행하면 됨, URI 주소는 기본적으로 현재의 로컬 IP 사용
노드	- ROS에서 실행되는 최소단위의 프로세스를 의미 - ROS에서는 하나의 목적에 하나의 노드 작성 권장 - 노드가 구동되면, 노드의 역할(토픽, 서비스 등), 메시지 형태 등이 등록됨

패키지	- ROS를 구성하는 기본 단위 - ROS 응용프로그램은 패키지 단위로 개발되며 최소한 하나의 노드를 가짐
메시지	- 노드와 노드는 서로 메시지를 통해서 데이터를 주고받음 - 메시지는 <code>int</code> , <code>float</code> , <code>boolean</code> 등의 기본 형태임 - 메시지 안에 메시지를 묶는 데이터 구조가 가능 - 단방향은 토픽(<code>topic</code>), 양방향은 서비스(<code>service</code>)
토픽	- 토픽은 ROS의 노드 사이의 데이터 통신 종류 - 단방향 통신으로, 데이터를 메시지에 담아서 방송함 - 방송을 하는 퍼블리셔(<code>publisher</code>)와 수신을 하는 서브스크라이버(<code>subscriber</code>)로 구성됨

4.2 ROS2와 DDS

• 산업 표준

- DDS는 OMG가 관리하는 산업 표준 미들웨어로, ROS2를 포함한 다양한 분야에서 활용되며 로봇 기술의 확장성을 높이고 있음

• 운영체제 독립

- DDS는 **Linux, Windows, macOS, Android, VxWorks** 등 다양한 운영체제 지원

• 언어 독립

- DDS는 미들웨어로서 프로그래밍 언어에 독립적이며, ROS2는 이를 기반으로 다양한 언어의 클라이언트 라이브러리를 제공해 멀티 언어를 지원함

• UDP 기반의 전송 방식

- ROS2는 DDS의 UDP 기반 멀티캐스트를 활용해 효율적인 통신을 구현하며, TCPROS 대비 유연성과 확장성이 향상되었고 QoS로 신뢰성 문제를 보완

• 데이터 중심적 기능

- DDS는 데이터 중심적(pub-sub) 구조로, 데이터의 형식과 전달 방식, 신뢰성 등을 미들웨어 수준에서 효율적으로 관리

• 동적 검색

- ROS2는 DDS의 동적 검색 기능을 통해 ROS1의 마스터 없이도 노드 간 자동 연결이 가능해져, 더 유연하고 안정적인 시스템 구성이 가능

• 확장 가능한 아키텍처

- DDS는 소형 디바이스부터 대형 인프라까지 확장 가능한 구조로, 복잡한 분산 시스템을 단일 표준 계층에서 간소화하여 ROS와 같은 대규모 로봇 시스템에 적합

- 상호 운용성

- ROS2에서 사용하는 DDS는 상호 운용성을 지원해 다양한 벤더 제품 간 통신이 가능

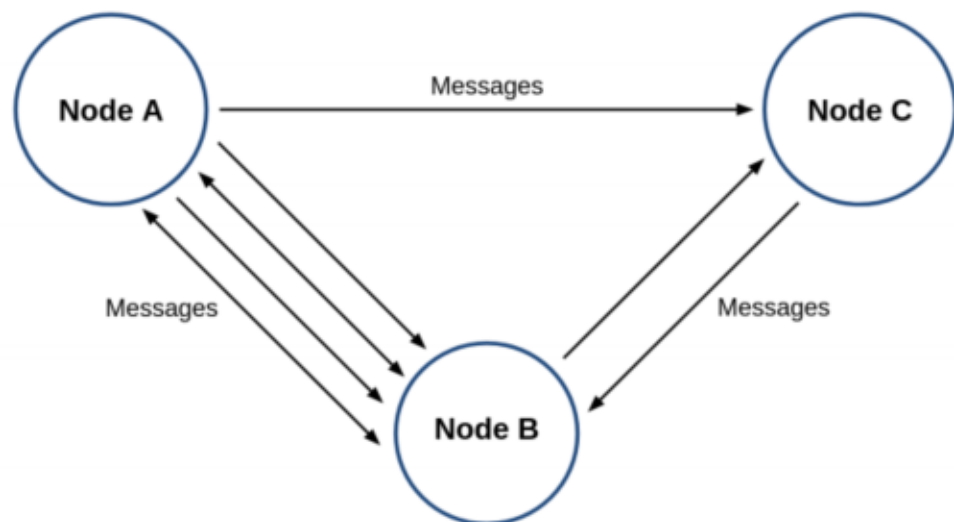
- 보안

- ROS2는 DDS-Security와 SROS2 도구를 통해 ROS1의 보안 취약점을 보완

4.3 ROS2 - 노드와 메시지 통신

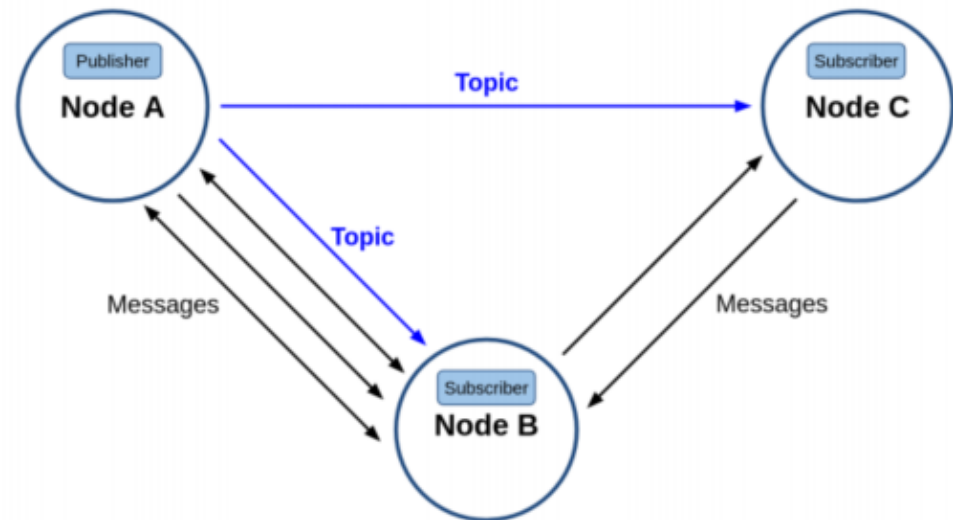
- Node

- 각각의 노드들은 서로 유기적으로 Message로 연결되어 사용됨
- 테스트가 많아질수록 메시지로 연결되는 노드가 늘어나며 시스템이 확장할 수 있게 됨



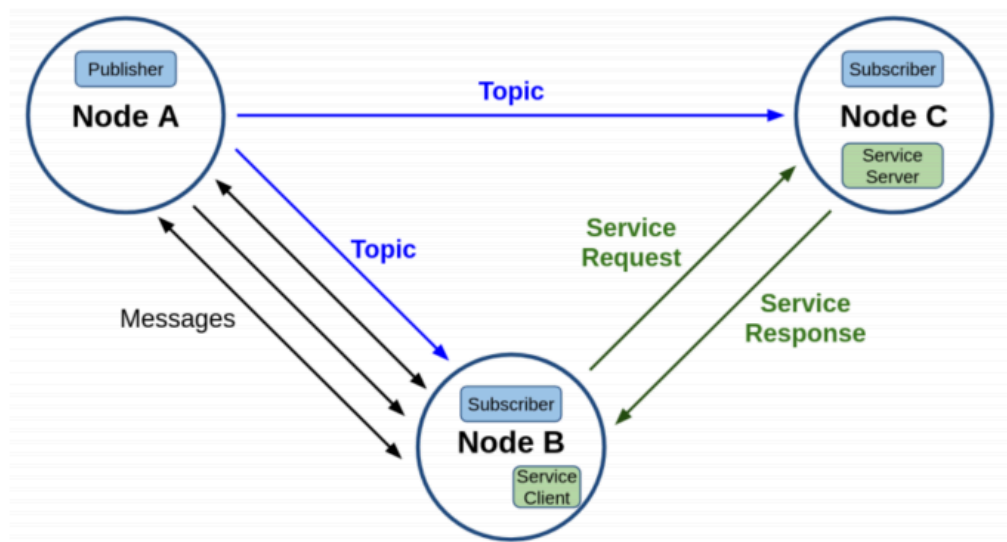
- Topic

- 비동기식 단방향 메시지 송수신 방식으로 메시지 형태의 메시지를 발간하는 Publisher와 메시지를 구독하는 Subscriber 간의 통신이라고 볼 수 있음



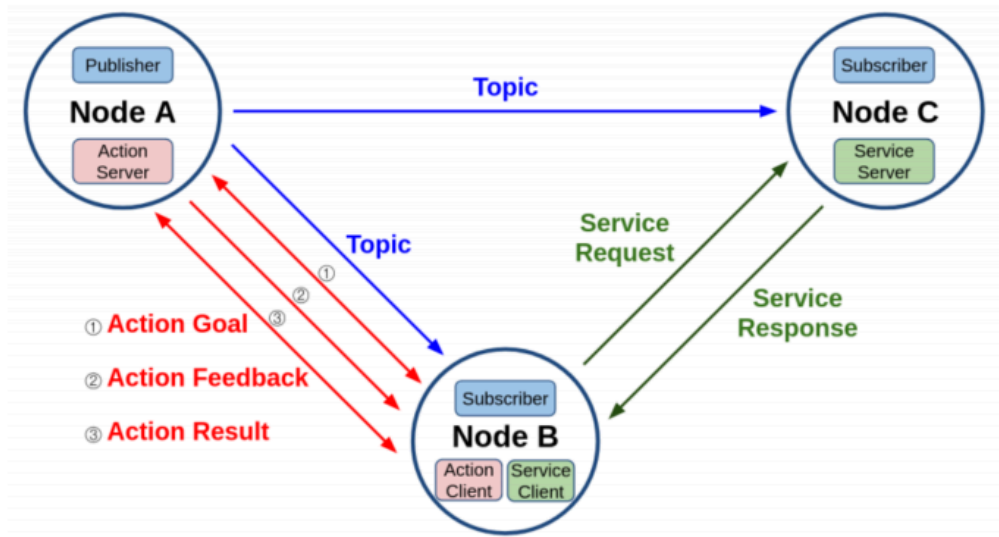
- **Service**

- 동기식 양방향 메시지 송수신 방식으로 서비스의 요청을 하는 쪽을 Service Client 라고 하며 서비스의 응답을 하는 쪽을 Service server라고 함



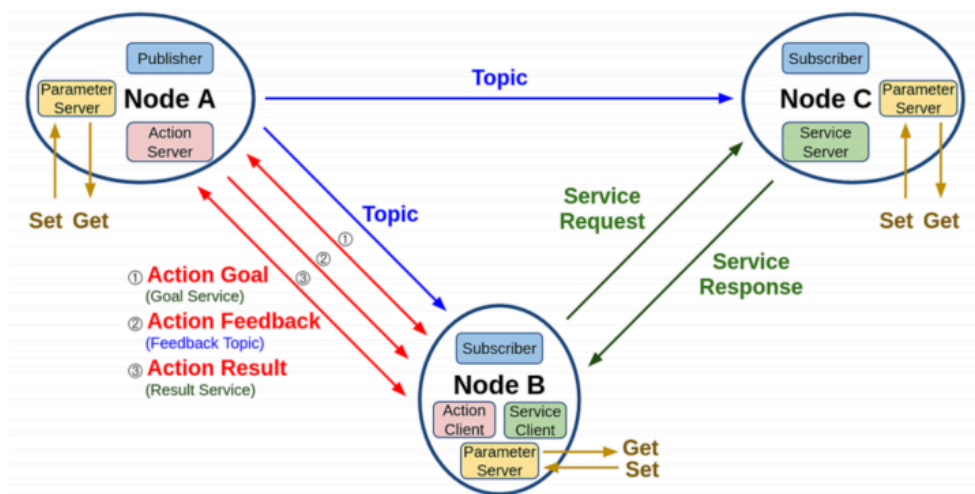
- **Action**

- 비동기식 + 동기식 양방향 메시지 송수신 방식



- **Parameter**

- 각 노드에 파라미터 관련 Parameter server를 실행시켜 외부의 Parameter client간의 통신으로 파라미터를 변경하는 것으로 서비스와 동일



- **토픽, 서비스, 액션**

	토픽 (topic)	서비스 (service)	액션 (action)
연속성	연속성	일회성	복합 (토픽+서비스)
방향성	단방향	양방향	양방향
동기성	비동기	동기	동기 + 비동기
다자간 연결	1:1, 1:N, N:1, N:N (publisher:subscriber)	1:1 (server:client)	1:1 (server:client)
노드 역할	발행자 (publisher) 구독자 (subscriber)	서버 (server) 클라이언트 (client)	서버 (server) 클라이언트 (client)
동작 트리거	발행자	클라이언트	클라이언트
인터페이스	msg 인터페이스	srv 인터페이스	action 인터페이스
CLI 명령어	ros2 topic ros2 interface	ros2 service ros2 interface	ros2 action ros2 interface
사용 예	센서 데이터, 로봇 상태, 로봇 좌표, 로봇 속도 명령 등	LED 제어, 모터 토크 On/Off, IK/FK 계산, 이동 경로 계산 등	목적지로 이동, 물건 파지, 복합 태스크 등

4.4 ROS2 패키지

- `ros2 node list`
 - 현재 실행중인 노드의 이름 표시
- `ros2 node info`
 - 현재 실행중인 노드의 정보 표시
- Cmake : 파일 빌드 환경설정 파일인 Makefile을 보다 쉽고 편리하게 작성
- `ros2 run` [패키지 이름] [노드 이름]
 - 특정 패키지의 특정 노드를 실행하는 명령
- `ros2 launch` [패키지 이름] [런치파일 이름]
 - 특정 패키지의 특정 launch 파일을 실행하는 명령
- `rqt_graph`
 - 토픽과 노드의 관계 그래프 표시

4.5 Alias 설정

- `alias`
 - 실행 때 마다 `source`명령을 수행하는 것은 비효율적이고 무척 귀찮은 작업
 - 미리 명령을 작성하여 이를 `alias`로 수행하는 방법으로 해결

```
alias humble='source /opt/ros/humble/setup.bash; echo \\\"ROS2 humble is activated.\\\"'  
alias sb='source ~/.bashrc'
```

```
spark@sparkk-750XDA:~$ humble  
"ROS2 humble is activated"
```

4.6 Turtlesim

- Turtlesim 설치

```
sudo apt update  
sudo apt install ros-humble-turtlesim
```

- 동작 확인

```
ros2 run turtlesim turtlesim_node  
ros2 run turtlesim turtle_teleop_key
```

- Turtlesim 패키지, 노드, 토픽, 서비스, 액션 조회

```
ros2 pkg list      # 설치된 패키지 목록 확인  
ros2 node list     # 현재 실행 중인 노드 목록 확인  
ros2 topic list    # 사용 가능한 토픽 목록 확인  
ros2 service list  # 사용 가능한 서비스 목록 확인  
ros2 action list   # 사용 가능한 액션 목록 확인  
rqt_graph          # 노드 간 관계 시각화 (GUI 도구)
```

5. ROS2 실습

5.1 Topic 실습


```

$ ros2 topic list -t # 현재 사용 가능한 모든 토픽을 유형(type)과 함께 출력

# 특정 토픽(/turtle1/cmd_vel)의 메시지 타입과 pub/sub 수 확인
$ ros2 topic info /turtle1/cmd_vel
Type:geometry_msgs/msg/Twist
Publisher count: 1
Subscriber count: 1

# 거북이 속도 데이터 확인용
$ ros2 topic echo /turtle1/cmd_vel
linear:
x:1.0
y:0.0
z:0.0
angular:
x:0.0
y:0.0
z:0.0

# 선형속도 x 방향 2.0, 회전속도 z 방향 1.8로 설정하여 한 번 발행
$ ros2 topic pub --once/turtle1/cmd_vel geometry_msgs/msg/Twist "{linear:
{x:2.0,y:0.0,z:0.0},angular:{x:0.0,y:0.0,z:1.8}}"

# 1초마다 거북이를 전진 및 회전시키는 명령을 반복 발행
$ ros2 topic pub --rate 1/turtle1/cmd_vel geometry_msgs/msg/Twist "{linea
r:{x:2.0,y:0.0,z:0.0},angular:{x:0.0,y:0.0,z:1.8}}"

```

5.2 Service 실습

```

# turtlesim 키보드 제어 노드 실행 (방향키로 거북이를 움직일 수 있음)
$ ros2 run turtlesim turtle_teleop_key

# turtlesim 배경과 거북이 선을 모두 초기화 (지우개 기능과 유사)
$ ros2 service call /clear std_srvs/srv/Empty

# "turtle1"이라는 이름의 거북이 제거 (죽이기 기능)

```

```
$ ros2 service call /kill turtlesim/srv/Kill "name:'turtle1'"

# turtlesim 전체 상태 초기화 (거북이 재배치 + 배경 리셋)
$ ros2 service call /reset std_srvs/srv/Empty

# 펜 설정: 색상(RGB 255,255,255) 및 선 굵기(width=10) 설정
$ ros2 service call /turtle1/set_pen turtlesim/srv/SetPen "{r:255,g:255,b:255,width:10}"
```

5.3 Action 실습

```
# 액션에 대한 정보 출력
$ ros2 action info /turtle1/rotate_absolute

# 액션 타입을 사용하여  $\theta$ (회전각도)를 1.5708 라디안(약 90도)으로 회전 지시
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.5708}"

# 목표 회전 도중의 Feedback 메시지를 실시간으로 확인할 수 있음
$ ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/RotateAbsolute "{theta: 1.57}" --feedback
```

5.4 Parameter get & set 실습

```
# 배경색(r,g,b)의 현재 값을 가져옴
$ ros2 param get /turtlesim background_r
$ ros2 param get /turtlesim background_g
$ ros2 param get /turtlesim background_b

# 배경색 값 설정
$ ros2 param set /turtlesim background_r 148
$ ros2 param set /turtlesim background_g 0
$ ros2 param set /turtlesim background_b 211

# 노드의 현재 파라미터를 모두 터미널에 출력
```

```
$ ros2 param dump /turtlesim
# 결과를 yaml파일에 저장
$ ros2 param dump /turtlesim > turtlesim_params.yaml

# turtlesim 노드를 실행
$ ros2 run turtlesim turtlesim_node
# 파라미터 파일을 적용하여 turtlesim 노드를 실행
$ ros2 run turtlesim turtlesim_node --ros-args --params-file ./turtlesim_params.yaml
```

5.5 rViz

- URDF - xml기반의 텍스트 파일
 - 로봇의 구조 정의
 - 로봇 동작 시각화
 - 로봇의 충돌 모델 정의
-